



Acceleration (kinda)

BY GARRETT DOOLITTLE & ETHAN HUTTON

State of the art

- ▶ As far as acceleration goes state of the art is GPU based ray tracing.
- ▶ This takes advantage of the thousands of cores/threads to do mass parallelization in the GPU and other hardware accelerations like RT cores that are specifically optimized for ray tracing.
- ▶ If you take GPUs out of the equation the state of the art on a CPU is using SIMD (single instruction, multiple data) vector-based calculations.
- ▶ We didn't do either of these*

What We accomplished

- ▶ StacklessBVH/RopedBVH
- ▶ Faster Ray Intersects
- ▶ Compressed Nodes
- ▶ OpenMP
- ▶ RayPackets
- ▶ Sphere Intersection (kinda)
- ▶ SIMD intersects (kinda)

Measuring Results

- ▶ All at 3000 x 3000 with -O3 flag and in Debug Mode
- ▶ Base ray tracer: 2.681s, 2.840s, 2.699s
 - ▶ Avg: 2.740s
- ▶ Base + Compressed Nodes: 2.847s, 2.844s, 2.829s
 - ▶ Avg: 2.838s
- ▶ Prev + New Intersects: 2.742s, 2.741s, 2.755s
 - ▶ Avg: 2.746s
- ▶ Prev + Ray Packets: 2.550s, 2.429s, 2.489s
 - ▶ Avg: 2.489s
- ▶ Prev + StackBVH: 2.212s, 2.221s, 2.226s
 - ▶ Avg: 2.220s
- ▶ Prev + OMP + Other Optimization Flags: .693s, .687s, .708
 - ▶ Avg: .696s

What did we learn

- ▶ Parallelize tasks when possible
- ▶ It is more efficient to not use recursion
- ▶ Intersect is where a lot of time is spent, so optimizing it even a little is worth it

Next steps

- ▶ Convert the Ray Packets to SIMD properly
- ▶ Do actual GPU acceleration

Questions?

- ▶ If you have no questions, ask Garrett how he implemented his BVH Tree in his Ray Tracer.